Description of the SOREL CAN Bus Interface (SCBI)

1. Goals

The development of the SCBI has the following goals:

1. Providing read access to the controller's state, i.e. useful internal variables.
   Example use case: datalogger

2. Supplying other CAN bus subscribers with information on preconfigured events.

   Examples:
   - Controller with ID=0x39 switched to “eco mode”
   - Controller with ID=0xFA requests additional heat

3. Control of other CAN bus devices

2. Technical Description

2.1 Overview

table 1: Structure of a SOREL CAN Package

Address ID 29 bits	bit 0..7: Program Type	Control program. Defines which actions can be performed and sets the context, in which they are performed.
	bit 8..15: Subscriber ID	unique CAN Subscriber ID of the sender
	bit 16..23: Function Type	action to perform Each Program Type provides a different set of actions.
	bit 24..26: Protocol Type	One for payloads <=8 Bytes and one for payloads >8 Bytes.
	bit 27..28: CAN Message Type	type of message Each Protocol Type has different Message Types.
Body 64 bits	bit 29..92: Payload	up to 8 Bytes of data per CAN package

Address ID
The SCBI is derived from the CAN-Open protocol. As the basis of the interface a 29 bit extended address resp. extended identifier (29bit Ext ID) is used. Address ID format for a specific “Protocol Type”:

Format: `ProgramType . SubscriberID . FunctionType . CANMessageType`

Example: `CONTROLLER . SubscriberID . I_AM_HERE . MSG_RESPONSE`

Protocol Type details
All current projects use the following two protocol types:

- CAN_FORMAT_0 Transmits a single CAN package with a payload of *up to 8 Bytes*
- CAN_FORMAT_BULK Transmits multiple CAN packages for payloads *larger than 8 Bytes*

The protocol type is encoded within bits 24 to 26 of the Address ID. Depending on which “Protocol Type” and “Program Type” is selected , the meaning of the remaining bits changes, i.e. for each Program Type a certain Function Type value can have a different meaning and depending on the Protocol Type, CAN Message Type values have a different meaning as well.

2.2 Protocol Type: (0) CAN_FORMAT_0

table 2: Structure of a CAN Package (Protocol Type: CAN_FORMAT_0)

Address ID 29 bits	bit 0..7: Program Type	0 CAN_PROGRAM_TYPE_UNDEFINED = 0 1 PROGRAM 2 OUTDOOR_SENSOR 3 CONSUMER 4 HEATING_MANAGER 5 ROOM_DEVICE 6 HEAT_SOURCE 7 HEAT_ACCUMULATOR 8 EXTENDED_CONTROLLER 9 EXTENSION_CONTROLLER 10 MONITORING_DEVICE 11 CONTROLLER 12 HEATING_CIRCUIT 13 SCHEDULE 14 MIXED_CIRCUIT 15 STRAIGHT_CIRCUIT 16 DHW 17 FLOW_THROUGH_DHW 18 BOILER 19 POOL 20 THERMOSTAT 21 REMOTE_CONTROL 0x80 DATALOGGER_MONITOR = 0x80 0x81 EVENT 0x82 FWC_CASCADE 0x83 REMOTESENSOR 0x83 DATALOGGER_NAMEDSENSORS = REMOTESENSOR (for compatibility) 0x84 HCC 0x87 CLOCKSYNC (send/ receive date and time) 0x88 REMOTERELAY
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	*action to perform*
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0** (1): CAN_FORMAT_BULK
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*data*

CAN_FORMAT_0 is used to transmit payloads smaller than 8 Byte.

CAN Message Types:

(0) MSG_REQUEST ...
(1) MSG_RESERVE ...
(2) MSG_RESPONSE ...
(3) MSG_ERROR …

2.2.1 Program Type: (11) CONTROLLER

table 3: Structure of a CAN Package (Program Type: CONTROLLER)

Address ID 29 bits	bit 0..7: Program Type	**(11): CONTROLLER**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	0 HAS_ANYBODY_HERE = 0 // discover CAN subscribers 1 I_AM_HERE = 1 2 GET_CONTROLLER_ID 3 GET_ACTIVE_PROGRAMS_LIST 4 ADD_PROGRAM 5 REMOVE_PROGRAM 6 GET_SYSTEM_DATE_TIME 7 SET_SYSTEM_DATE_TIME 8 I_AM_RESETED 9 DATALOGGER_TEST = 9 10 FWC_CASCADE0 19 FWC_CASCADE0_END = FWC_CASCADE0+9 (10 functions reserved for FWC cascading)
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0** (1): CAN_FORMAT_BULK
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*see individual description below*

(0) HAS_ANYBODY_HERE
On the first start, at which point the controller's CAN Subscriber ID is still undefined, it sends this message:

```
* Address ID: CONTROLLER.SubscriberID.HAS_ANYBODY_HERE.MSG_RESPONSE
* Body      : (0)
```

Every controller listening on the CAN bus responds with the message I_AM_HERE. Since this message includes the CAN Subscriber ID of each responding controller, this exchange allows the new controller to choose an unused Subscriber ID.

(1) I_AM_HERE
Every 60 seconds each controller sends the message I_AM_HERE to every other controller.

```
* Address ID: CONTROLLER.SubscriberID.I_AM_HERE.MSG_RESPONSE
* Body      : CONTROLLER_ID.CFG_DEVICE_ID.CFG_OEM_ID.CFG_DEVICE_VARIANT
```

(8) I_AM_RESETED
This message is send, as soon as a controller completed a restart and already has a CAN ID.

```
* Address ID: CONTROLLER.SubscriberID.I_AM_RESETED.MSG_RESPONSE
* Body      : (0)
```

(9) DATALOGGER_TEST
A series of messages needed to carry out the hardware test of the datalogger.

2.2.2 Program Type: (0x80) DATALOGGER_MONITOR

table 4: Structure of a CAN Package (Program Type: DATALOGGER_MONITOR)

Address ID 29 bits	bit 0..7: Program Type	**(0x80): DATALOGGER_MONITOR**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	0 DLF_UNDEFINE = 0 1 DLF_SENSOR 2 DLF_RELAY 3 DLG_HYDRAULIK_PROGRAMM 4 DLG_ERROR_MESSAGE 5 DLG_PARAM_MONITORING 6 DLG_STATISTIC 7 DLG_OVERVIEW 8 DLG_HYDRAULIK_CONFIG
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0** (1): CAN_FORMAT_BULK
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*see individual description below*

(3) DLG_HYDRAULIK_PROGRAMM
Request the number of the currently selected control program from a specific controller via its ContollerID.

```
* Address ID: DATALOGGER_MONITOR.SubscriberID.DLG_HYDRAULIK_PROGRAMM.MSG_REQUEST
* Body      : (0)
```

(3) DLG_HYDRAULIK_PROGRAMM
Response containing the control program number

```
* Address ID: DATALOGGER_MONITOR.SubscriberID.DLG_HYDRAULIK_PROGRAMM.MSG_RESPONSE
* Body      : [max. 8 characters]   the current program's name
```

(8) DLG_HYDRAULIK_CONFIG
Controller parameter: currently selected control program number, number of relays and sensors

```
* AddressID: DATALOGGER_MONITOR.SubscriberID.DLG_HYDRAULIK_CONFIG.MSG_RESPONSE
* Body     : N_PHYSICAL_SENSORS, N_RELAIS_PARAMS, ...
```

(1) DLF_SENSOR
Contains the current sensor value

```
* Address ID: DATALOGGER_MONITOR.SubscriberID.DLF_SENSOR.MSG_RESPONSE
* Body      : SensorNo.Value
      - SensorNo  [1 Byte]
      - value     [32 bit integer]
```

(2) DLF_RELAY

Contains the current relay value

```
* Address ID: DATALOGGER_MONITOR.SubscriberID.DLF_RELAY.MSG_RESPONSE
* Body:
      - RelayNo [1 Byte]

      - mode    [1 Byte]
            0  RELAISMODE_SWITCHED     // on/off
            1  RELAISMODE_PHASE        // phase angle speed control
            2  RELAISMODE_PWM
            3  RELAISMODE_VOLTAGE

      - value   [1 Byte]

      - ExFunction: extra function connected to the relay
           -2  EF_DISABLED    = -2
           -1  EF_UNSELECTED  = -1

            0  EF_SOLARBYPASS = 0
            1  EF_HEIZEN
            2  EF_HEIZEN2
            3  EF_KUEHLEN
            4  EF_RUECKLAUFANHEBUNG
            5  EF_DISSIPATION
            6  EF_ANTILEGIO
            7  EF_UMLADUNG
            8  EF_DIFFERENZ
            9  EF_HOLZKESSEL

           10  EF_SCHUTZFUNKTION
           11  EF_DRUCKREGELUNG
           12  EF_BOOSTER
           13  EF_R1PARALLELBETRIEB
           14  EF_R2PARALLELBETRIEB
           15  EF_DAUEREIN
           16  EF_HEIZKREISRC21
           17  EF_CIRCULATION
           18  EF_STORAGEHEATING
           19  EF_STORAGESTACKING

           20  EF_R_V1_PARALLEL
           21  EF_R_V2_PARALLEL
           22  EF_R1_PERMANENTLY_ON
           23  EF_R2_PERMANENTLY_ON
           24  EF_R3_PERMANENTLY_ON = 24

           25  EF_V2_PERMANENTLY_ON = 25
           26  EF_EXTERNALALHEATING
           27  EF_NEWLOGMESSAGE

           28  EF_EXTRAPUMP
           29  EF_PRIMARYMIXER_UP
           30  EF_PRIMARYMIXER_DOWN
           31  EF_SOLAR
           32  EF_CASCADE
```

(3) DLG_HYDRAULIK_PROGRAMM

…

```
* Address ID: ...
* Body:
      - HydraulicProgramName  [maximum of 8 Bytes]
```

(4) DLG_ERROR_MESSAGE

Contains MessageLogEntry.
This is a string. Strings are always transmitted using CAN_FORMAT_BULK.

(5) DLG_PARAM_MONITORING

Complete parameter information (network name, value, min value, max value, …)
Transmitted via CAN_FORMAT_BULK.

(6) DLG_STATISTIC

…

```
* Address ID: ...
* Body:
      - index   [1 Byte]
             0  BETRIEB_ART = 0
             1  LEISTUNG
             2  LEISTUNG_VFS1
             3  LEISTUNG_VFS2
             4  ERTRAG_CONST         // Ertrag == heat yield
             5  ERTRAG_VFS1
             6  ERTRAG_VFS2
             7  ERTRAG_TOTAL
             8  WMZ_AKTIVE
             9  VFS1
            10  VFS2

      - value   [32 bit integer]
```

(7) DLG_OVERVIEW

…

```
* Address ID: ...
* Body:
      - type   [bits 8 to 6 of byte[0]]
            1  DAYS = 1
            2  WEEKS
            3  MONTHS
            4  YEARS
            5  TOTAL
            6  STATUS

      - index       [bits 5 to 1 of byte[0]]
      - hours       [16 bit integer]
      - heat yield  [32 bit integer]
```

(8) DLG_HYDRAULIK_CONFIG

…

```
* Address ID: ...
* Body:
      - MAX_Sensors        [1 Byte]
      - MAX_Relay          [1 Byte]
      - TemperatureFormat  [1 Byte]
```

2.2.3 Program Type: (0x81) EVENT

table 5: Structure of a CAN Package (Program Type: EVENT)

Address ID 29 bits	bit 0..7: Program Type	**(0x81): EVENT**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	(1): EFT_SOLAR (2): EFT_EXFUNC
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0** (1): CAN_FORMAT_BULK
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*see individual description below*

(1) EFT_SOLAR

…

```
* Body:
      - function    [1 Byte]
            0  SF_DISSIPATION
            1  SF_ANLAGENSCHUTZ
            2  SF_FROSTSCHUTZ
            3  SF_STARTHILFE
            4  SF_KOLLEKTORSCHUTZ
            5  SF_PROGRAMM

      - RelayIndex  [1 Byte] index of relay in use by respective function
      - value       [1 Byte] function state (on/off) or relay value
```

(2) EFT_EXFUNC

…

```
* Body:
      - function    [1 Byte]
        ExFunction values: DATALOGGER_MONITOR.SubscriberID.DLF_RELAY.MSG_REQUEST
      - RelayIndex  [1 Byte] index of relay in use by respective function
      - value       [1 Byte] function state (on/off) or relay value
```

2.2.4 Program Type: (0x82) FWC_CASCADE

Freshwater controller cascading functions

table 6: Structure of a CAN Package (Program Type: FWC_CASCADE)

Address ID 29 bits	bit 0..7: Program Type	**(0x82): FWC_CASCADE**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	10 ecfControl = 10 11 ecfSettings1 // tSet, Tmax, alTset, ... 12 ecfSettings2 // AL function's times 13 ecfSettings3 // circulation settings 14 ecfSettings4 // circulation function's times 15 ecfSettings5 // time and date 16 ecfSensors // sensor values 17 ecfCommand // add/ remove station
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0** (1): CAN_FORMAT_BULK
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*see individual description below*

(10) ecfControl

Main command used to control the cascade

```
* Address ID: FWC_CASCADE.FWC_Cascade_SubscriberID.ecfControl.MSG_REQUEST
* Body:
      - currentFlow  [2 Bytes]
            value of the flow sensor (DE: Durchflussfühler)

      - maxFlow      [2 Bytes]
            maximum flow rate that can be measured
            depends on the selected flow sensor type

      - flags        [1 Byte]
               1  CASCADEF_FIXBASE
                  This station is the fixed base, i.e. it is always active.

               4  CASCADEF_NEEDHELP
            This station is requesting help

               8  CASCADEF_WANTCLOSEVALVE
            This station wants to close its valve

            0x10  CASCADEF_TSETLIMIT
                  Tset cannot be reached

            0x80  CASCADEF_VALVEOPEN
            This stations valve is open

      - relay state        [1 Byte]   current circulation relay state
        (0 .. 255)

      - unused             [1 Byte]
        (0)

      - CAN Subscriber ID  [1 Byte]
        (1 .. 255)
```

(11) ecfSettings1
Synchronize some config values

```
* Address ID: FWC_CASCADE.FWC_Cascade_SubscriberID.ecfSettings1.MSG_REQUEST
* Body:
      - Tset          [2 Bytes]  (DE: Tsoll)
      - Tmax          [2 Bytes]
      - AlTset        [2 Bytes]
      - AlActionTime  [1 Byte]   Al application time in minutes
      - AlFunction    [1 Byte]   Al function active (0,1)
```

(12) ecfSettings2
Sychronize the Anti-Legionella function's times

```
* Address ID: FWC_CASCADE.FWC_Cascade_SubscriberID.ecfSettings2.MSG_REQUEST
* Body:
      - start  [2 Bytes]    start time
      - end    [2 Bytes]    end   time
      - entry  [1 Byte]     the day's entry number in the list
```

(13) ecfSettings3
Synchronize the circulation settings

```
* Address ID: FWC_CASCADE.FWC_Cascade_SubscriberID.ecfSettings3.MSG_REQUEST
* Body:
      - CircTmin  [2 Bytes]
      - CircHyst  [2 Bytes]  hysteresis
      - CircFmax  [1 Byte]   maximum flow
      - CircMode  [1 Byte]
      - unused    [1 Byte]
      - CasDelay  [1 Byte]   cascades' delay in seconds
```

(14) ecfSettings4
Synchronize the circulation function's times

```
* Address ID: FWC_CASCADE.FWC_Cascade_SubscriberID.ecfSettings4.MSG_REQUEST
* Body:
      - start  [2 Bytes]    start time
      - end    [2 Bytes]    end   time
      - entry  [1 Byte]     the day's entry number in the list
```

(15) ecfSettings5
Synchronize time and date

```
* Address ID: FWC_CASCADE.FWC_Cascade_SubscriberID.ecfSettings5.MSG_REQUEST
* Body:
      - year    [2 Bytes]
      - month   [1 Byte]
      - day     [1 Byte]
      - hour    [1 Byte]
      - minute  [1 Byte]
      - second  [1 Byte]
      - isDst   [1 Byte]   flag for summer time
```

(16) ecfSensors

Synchronize sensor values, in case the message's reciever lacks a corresponding sensor

```
* Address ID: FWC_CASCADE.FWC_Cascade_SubscriberID.ecfSensors.MSG_REQUEST
* Body:
      - Sensor 1                    [2 Bytes]   sensor value
      - Sensor 2                    [2 Bytes]   sensor value
      - CAN Subscriber ID Sensor 1  [1 Byte]    controller that provided sensor 1
      - CAN Subscriber ID Sensor 2  [1 Byte]    controller that provided sensor 2

      - index                       [1 Byte]
        (0): sensor 1 and 2
        (1): sensor 3 and 4

      - unused                      [1 Byte]
```

(17) ecfCommand

Add or remove stations (open/ close the valve)

```
* Address ID: FWC_CASCADE.FWC_Cascade_SubscriberID.ecfCommand.MSG_REQUEST
* Body:
      - command         [1 Byte]
        (0): add    station  (open  the valve)
        (1): remove station  (close the valve)

      - target station  [1 Byte]
        CAN Subscriber ID of the device intended to carry out the command

      - alManually      [1 Byte]
```

2.2.5 Program Type: (0x83) REMOTESENSOR

Request sensor values by name, e.g. outdoor, room temp/ humidity, RC switch, or get changes

table 7: Structure of a CAN Package (Program Type: REMOTESENSOR)

Address ID 29 bits	bit 0..7: Program Type	**(0x83): REMOTESENSOR**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*

	bit 16..23: Function Type	// CAN: must be fix
		(0): ecsOutdor
		(1): ecsFlow
		(2): ecsCold
		(3): ecsSolar
		(4): ecsReserved1
		(5): ecsReserved2
		(6): ecsReserved3
		(7): ecsStorage
		(8): ecsStorageTop
		(9): ecsStorageCenter
		(10): ecsStorageBottom
		(11): ecsReserved4
		// room control
		(12): ecsRcTset
		(13): ecsRcHumidity
		(14): ecsRcTroom
		(15): ecsRcWheel
		(16): ecsRcSwitch

	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0** (1): CAN_FORMAT_BULK
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	MSG_REQUEST: payload 0 Byte long MSG_RESPONSE: payload 4 Bytes long data[0]: value low Byte data[1]: value high Byte data[2]: virtual circuit (Caleon)/ room controller unit (XHCC) starting with “0” data[3]: type of room controller; see “namedSensors.h”

2.2.6 Program Type: (0x84) HCC

Heating circuit control program, e.g. heat request

table 8: Structure of a CAN Package (Program Type: HCC)

Address ID 29 bits	bit 0..7: Program Type	**(0x84): HCC**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	(0): HEATREQUEST (1): HEATINGCIRCUIT_STATE
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0** (1): CAN_FORMAT_BULK
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*see individual description below*

(0) HEATREQUEST

…

```
* Address ID: HCC.SubscriberID.HCC_HEATREQUEST.MSG_REQUEST
              HCC.SubscriberID.HCC_HEATREQUEST.MSG_RESPONSE
* Body:
      - heatRequest [1 Byte]
            The requested flow temperature (DE: VL-Temperatur) is transmitted as a
            relay value in the same way as it would be represented on a 0..10 V
            output in the "Modulate" mode. Therefore a temperature between 0 to
            100 °C will be represented as 0 to 100 % of a total of 255, e.g. a
            temperature of 53 °C results in a value of 135. If no heat shall be
            requested, the value is 0. All controllers send their current value
            once per minute.

      - SolarWood   [1 Byte]
            Locally active energy source
            (0): solar load
            (1): solid fuel (wood) boiler
```

(1) HEATINGCIRCUIT_STATE

MSG_RESPONSE
Send current heating circuit state if the state changed but not more than once every 10 seconds per circuit.

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE.MSG_RESPONSE
* Body:
      [8 Bytes] payload
      - circuit   [1 Byte]
            circuit index
            (0)..(255)

      - state     [1 Byte]
            teRcModes from roomController.h

      - tFlowSet  [2 Bytes]
            temperature value calculated flow

      - tFlow     [2 Bytes]
            temperature value current flow

      - tStorage  [2 Bytes]
            temperature value of storage
```

MSG_REQUEST
Request heating circuit state for all circuits

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE.MSG_REQUEST
* Body:
      [0 Byte] payload
```

2.2.7 Program Type: (0x88) REMOTERELAY

Set relay of another controller

table 9: Structure of a CAN Package (Program Type: REMOTETELAY)

Address ID 29 bits	bit 0..7: Program Type	**(0x88): REMOTERELAY**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	*unique CAN Subscriber ID of the reciever (!)*
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0** (1): CAN_FORMAT_BULK
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	MSG_REQUEST: payload 3 Bytes long data[0]: relay index (0) … (CFG_REMOTERELAY_COUNT-1) data[1]: new relay state (0): RELAIS_OFF (255): RELAIS_ON data[2]: bitmask with relays used by this function MSG_RESPONSE: payload 0 Byte long

2.3 Protocol Type: (1) CAN_FORMAT_BULK

table 10: Structure of a CAN Package (Protocol Type: CAN_FORMAT_BULK)

Address ID 29 bits	bit 0..7: Program Type	(0x80): bulk_DATALOGGER_MONITOR
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	*action to perform*
	bit 24..26: Protocol Type	(0): CAN_FORMAT_0 **(1): CAN_FORMAT_BULK**
	bit 27..28: CAN Message Type	(0): MSG_HEADER (1): MSG_DATA (2): MSG_RETRY (3): MSG_CONFIRM
Body 64 bits	bit 29..92: Payload	*data*

CAN_FORMAT_BULK is used to transmit payloads larger than 8 Bytes. The data is sent block wise. Each block consists of 8 Byte long lines. The first Byte represents the line index, comparable to an array, and indicates the line's position/ offset relative to the start of the block. The remaining 7 Bytes contain the data.

CAN Message Types:

(0) MSG_HEADER

…

```
* Body:
      - size       [32 bit integer]   data size in Bytes
      - offset     [32 bit integer]   starting position of data block in Bytes
```

(1) MSG_DATA

…

```
* Body:
      - LineIndex  [1 Byte]           index inside the data block
      - data       [7 Bytes]
```

(2) MSG_RETRY

Ask another CAN subscriber to resend a missing/ lost block

```
* Body:
      - offset     [32 bit integer]   starting position of data block in Bytes
      - LineIndex  [1 Byte]           index inside the data block
```

(3) MSG_CONFIRM

…

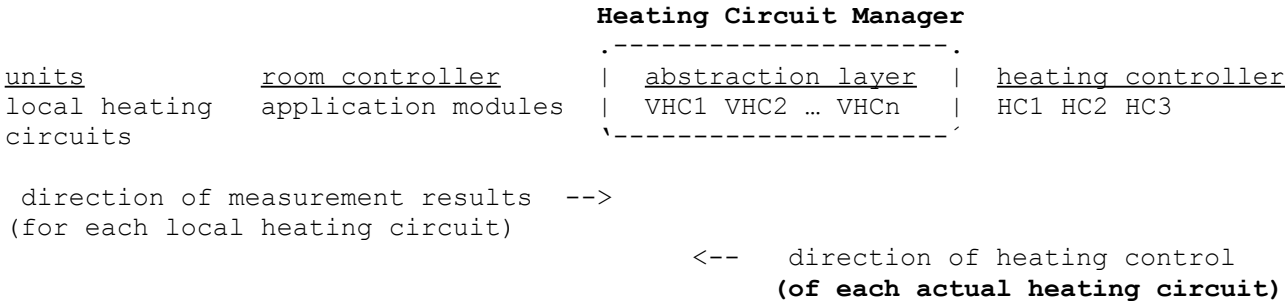
```
* Body:
      - size       [32 bit integer]   data size in Bytes
      - offset     [32 bit integer]   starting position of data block in Bytes
```

3 Heating Circuit Manager (HCM)

3.1 Overview

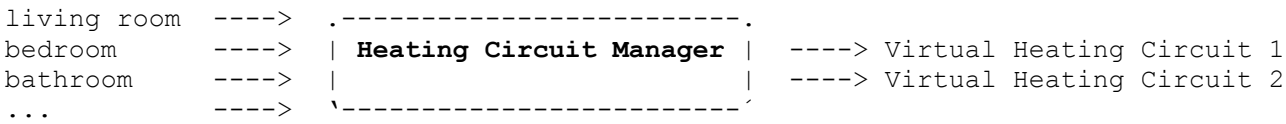
This section is a short description of the communication concept used to connect heating controllers (e.g. XHCC) with room controllers (Caleon).

The goal is to minimize the system's overall complexity while still providing enough flexibility to implement 99% of all use cases. This is done by introducing "virtual heating circuits" as an abstraction layer between the room controllers and heating controllers, i.e. the application modules and actual heating circuits.



On the input side the room controller's application modules handle individual groups of sensors/ rooms, referred to by us as "units" or "local heating circuits". In the room controller one or more such unit are assigned to a Virtual Heating Circuit (VHC). The heating controller on the other hand only sees the available virtual heating circuits and links them to actual Heating Circuits (HC). Each VHC provides an aggregated mode and temperature to the heating controller. Room controllers and heating controllers are connected via CAN.

The Heating Circuit Manager manages the assignment of each local heating circuit to a virtual heating circuit:



3.2 Influence of Room Controllers on a Heating Controller and its connected Heating Circuits

Every room controller continuously transmits Tset and Tact to the HCC. The HCC determines for each heating circuit the value pair with the largest difference. This difference is then used as the room influence (Raumeinfluss) for calculating the reference flow temperature (VL-Temp) for the respective heating circuit.

Every room controller transmits its operating mode (off, eco, regular, turbo) to the heating controller. The heating controller, however, continuous to use the operating mode derived from its own time schedule, unless every connected room controller transmits the same operating mode---only then the heating controller changes its operating mode.
Special case: Only one room controller is connected to the heating controller. In this case, this one room controller is equivalent to "all room controllers" and the heating controller works as described above.

The time schedule of the heating controller continues to exist, to be used, and to be displayed, if one or more room controllers with their own time schedule are connected.

Every time a room controller's operating mode changes due to the programmed time schedule, the new operating mode is transmitted to the heating circuit. Note: It makes no difference, which event caused the operating mode to change---user interaction or time schedule---the effect is the same.

4 Appendix

Revision History

Date	Rev.	Description	Name
2016-05-06	1	Translate EB's document version to English	MKM
2016-05-10	2	Clarify several details	MKM
2016-05-10	3	Add description of "Heating Circuit Manager" and "REMOTESENSOR"	MKM
2016-05-11	4	Add description of "REMOTERELAY"	MKM
2016-06-17	5	Add new Function Type "HEATINGCIRCUIT_STATE" Fix order of elements in the address field description (cp. 2.1)	MKM
2016-06-20	6	Add an overview for every Program Type Fix minor inconsistencies	MKM
2016-06-20	7	Make the technical description's overview easier to understand	MKM